

TEC/LD 控制与驱动板固件功能规划

这份文档从嵌入式软件角度规划这块板应该具备的功能。目标不是只写“能点亮、能发 PWM”，而是把它做成一块不会轻易烧 LD、不会让 TEC 乱冲、故障可诊断、参数可校准的控制板。

1. 软件总体职责

这块板的软件至少要负责 8 件事：

- 1. 电源和外设上电顺序控制。
- 2. SPI 管理 DAC8562 和 ADS1120。
- 3. DAC 输出模拟设定值。
- 4. ADC 采集电流、电压、温度等反馈。
- 5. TEC H 桥 PWM 控制和方向控制。
- 6. LD/Buck 输出软启动、限流和保护。
- 7. TEC 温度闭环控制。
- 8. 故障检测、锁定、日志和通信。

推荐软件分层：

Application	
-> control_tec.c	温度 PID、TEC 电流/方向规划
-> control_ld.c	LD 电流目标、软启动、限流
-> safety.c	故障检测、保护锁定、恢复策略
-> command.c	上位机命令、参数读写
Service	
-> measurement.c	ADC 原始值到物理量转换
-> calibration.c	零点、增益、NTC 参数、保存/加载
-> power_seq.c	上电、关断、使能时序
-> logger.c	事件、故障、运行数据记录
Driver	
-> drv_ads1120.c	ADC 驱动
-> drv_dac8562.c	DAC 驱动
-> drv_pwm.c	PWM、互补输出、死区、刹车
-> drv_gpio.c	使能、状态脚、LED
-> drv_uart.c	通信
-> drv_flash.c	参数存储

2. 系统状态机

不要让板子上电后直接输出。建议做显式状态机。

OFF	
-> INIT	
-> SELF_TEST	

```
-> READY  
-> RAMP_UP  
-> RUN  
-> RAMP_DOWN  
-> READY
```

任意状态 -> FAULT_LATCHED

FAULT_LATCHED -> OFF 或 CLEAR_FAULT -> SELF_TEST

2.1 OFF

输出全关：

- DAC 输出安全值。
- PWM 关闭。
- H 桥所有 MOSFET 关闭。
- LD Buck 使能关闭。
- 外部输出可选择关闭。

2.2 INIT

初始化：

- 时钟。
- GPIO 默认态。
- PWM 默认关断态。
- SPI。
- UART。
- ADC/DAC。
- 看门狗。
- 参数读取。

所有功率输出必须在初始化前就是硬件安全态。软件初始化之前不能依赖 MCU 输出某个电平才安全。

2.3 SELF_TEST

自检内容：

- 5V0/VCC/AVCC/REF2V5/-1V0 是否在合理范围。
- ADS1120 是否能通信。
- DAC8562 是否能通信。
- NTC 是否开路/短路。
- 电流采样零点是否离谱。
- 输出端是否短路或开路。
- 上一次是否有未清除故障。

2.4 READY

系统待命：

- 外设在线。
- 输出仍为 0。
- 可以接收设定值。
- 可以开始 ramp。

2.5 RAMP_UP

不能一步写满 DAC 或 PWM。要斜坡：

```
target = user_target
setpoint += slew_rate * dt
```

每次 ramp 都要检查：

- 电流是否跟随。
- 电压是否异常。
- 温度是否异常。
- 是否超过功率限制。

2.6 RUN

运行状态执行闭环：

- TEC 温度 PID。
- LD 电流/功率调节。
- ADC 周期采样。
- 故障检测。
- 通信上报。

2.7 FAULT_LATCHED

一旦进入故障锁定：

- 立即关 PWM。
- DAC 输出安全值。
- Buck/H 桥使能关闭。
- 记录故障码、时间、关键测量值。
- 等待用户明确清故障。

不要自动反复重启功率输出，否则可能把硬件推向热损坏。

3. DAC8562 驱动功能

DAC 驱动要包含：

- 初始化序列。
- 写 A 通道。
- 写 B 通道。
- 同步更新。

- 输出清零。
- 读回如果芯片支持则实现，不支持则用软件影子寄存器。
- 通信超时处理。

DAC 代码换算：

```
Code = round(Vout / Vref * 65536)
```

边界限制：

```
if Code < 0: Code = 0
if Code > 65535: Code = 65535
```

建议不要让应用层直接写 Code。应用层应该写物理量，例如：

```
dac_set_voltage(channel, voltage)
ld_set_current_ampere(current)
tec_set_current_ampere(current)
```

底层再做换算。

4. ADS1120 驱动功能

ADS1120 不是简单 read_adc() 就够了。建议实现：

- 寄存器配置。
- 通道选择。
- PGA 增益选择。
- 数据率选择。
- 参考源选择。
- 单次转换。
- 连续转换。
- DRDY 中断。
- 超时。
- CRC/一致性检查，如果协议支持。
- 原始码转有符号整数。

每个 ADC 通道都要有配置表：

通道	物理量	输入方式	PGA	参考	采样率	滤波
CH_CURRENT_A	电流 A	差分/单端	待确认	REF2V5	中速	IIR
CH_CURRENT_B	电流 B	差分/单端	待确认	REF2V5	中速	IIR
CH_NTC	温度	单端	1	REF2V5	低速	均值/IIR

通道	物理量	输入方式	PGA	参考	采样率	滤波
CH_VOLTAGE	输出电压	单端	1	REF2V5	中速	IIR

一阶 IIR 滤波：

$$y[n] = y[n-1] + \alpha * (x[n] - y[n-1])$$

其中：

$$\alpha = dt / (\tau + dt)$$

温度可以滤得慢一点，电流保护要快一点。

5. 测量换算层

不要在控制算法里到处写 ADC 公式。应该集中到 `measurement.c`。

5.1 电流换算

硬件近似：

$$\begin{aligned} R_{\text{shunt}} &= 1\text{m}\Omega \\ \text{Gain} &= 150 \\ V_{\text{amp}} &= \text{Gain} * I * R_{\text{shunt}} + V_{\text{offset}} \end{aligned}$$

反推：

$$I = (V_{\text{amp}} - V_{\text{offset}}) / (\text{Gain} * R_{\text{shunt}})$$

代入：

$$\begin{aligned} I &= (V_{\text{amp}} - V_{\text{offset}}) / (150 * 0.001) \\ &= (V_{\text{amp}} - V_{\text{offset}}) / 0.15 \end{aligned}$$

也就是：

$$1 \text{ V} \rightarrow 6.667 \text{ A}$$

实际固件不要硬编码 150。建议用校准参数：

```
current = (adc_voltage - current_offset_v) * current_gain_a_per_v;
```

其中 `current_gain_a_per_v` 通过标定得到。

5.2 NTC 换算

推荐参数：

```
typedef struct {  
    float r_fixed;  
    float r0;  
    float beta;  
    float t0_kelvin;  
    bool ntc_to_ground;  
} ntc_cal_t;
```

换算流程：

```
ADC code -> Vntc  
Vntc -> Rntc  
Rntc -> Kelvin  
Kelvin -> Celsius
```

必须检测：

```
Vntc < V_short_threshold -> NTC 短路  
Vntc > V_open_threshold -> NTC 开路
```

5.3 电压换算

如果输出电压经过分压：

```
Vadc = Vout * Rbot / (Rtop + Rbot)  
Vout = Vadc * (Rtop + Rbot) / Rbot
```

同样建议用校准参数：

```
vout = adc_voltage * voltage_gain + voltage_offset;
```

6. TEC 控制功能

TEC 控制不只是 PWM。它需要温度目标、方向、电流限制和热保护。

6.1 控制目标

常见模式：

1. 恒温模式：目标温度 T_{set} 。
2. 恒电流模式：直接设定 TEC 电流，调试用。
3. 手动 PWM 模式：只用于实验，不建议开放给普通用户。

6.2 温度 PID

基础 PID：

```
error = Tset - Tmeas
u = Kp*error + Ki*integral(error) + Kd*d(error)/dt
```

但 TEC 可以加热也可以制冷，所以 u 可以为正或负：

```
u > 0 -> 一个方向
u < 0 -> 反方向
abs(u) -> 电流目标或 PWM 占空比
```

必须加：

- 输出限幅。
- 积分限幅。
- 积分抗饱和。
- 方向切换斜坡。
- 温度采样滤波。

抗积分饱和伪代码：

```
float u_unsat = kp * err + integral + kd * derr;
float u_sat = clamp(u_unsat, -limit, limit);

if (u_unsat == u_sat || sign(err) != sign(u_unsat - u_sat)) {
    integral += ki * err * dt;
}
```

6.3 TEC 方向切换策略

绝不能在大电流下直接反向。

推荐：

```
if sign(new_current) != sign(old_current):  
    ramp current to 0  
    disable H bridge  
    wait deadtime + current_decay_time  
    switch direction  
    ramp current up
```

软件保护：

- 同一半桥不能高低边同时开。
- `TEC_PWM` 和 `TEC_PWMN` 不能同时使能错误状态。
- 方向切换要有状态机，不要一行代码改 GPIO。

6.4 PWM 配置

必须用硬件 PWM 外设生成互补波形和死区。

配置项：

- PWM 频率。
- 死区时间。
- 极性。
- 刹车输入。
- 上电默认关闭。
- 故障时硬件自动拉低输出。

如果用 STM32G431，高级定时器适合做这件事。

7. LD 控制功能

如果 `LD+` / `LD-` 真的接激光二极管，软件保护要非常保守。

7.1 LD 输出模式

建议支持：

- 恒流模式。
- 禁止直接恒压模式，除非只用于假负载测试。
- 软启动模式。
- 输出关闭并放电。

LD 电流目标应经过斜坡：

```
I_cmd += slew_rate * dt
```

7.2 LD 保护

必须有：

- 最大电流限制。
- 最大电压限制。
- 最大功率限制。
- 开路检测。
- 短路检测。
- 过温降额。
- 输出异常纹波检测，如果采样速度允许。

功率限制：

$$P_{out} = V_{out} * I_{out}$$

如果：

$$P_{out} > P_{limit}$$

则降低设定或关断。

7.3 开路和短路判断

开路可能表现：

I_{out} 很小
 V_{out} 升高到上限

短路可能表现：

I_{out} 快速升高
 V_{out} 很低

伪代码：

```
if (enable && iout < i_min && vout > v_open_threshold) {  
    fault_set(FAULT_LD_OPEN);  
}  
  
if (enable && iout > i_short_threshold && vout < v_short_threshold) {  
    fault_set(FAULT_LD_SHORT);  
}
```

8. 安全保护系统

建议故障分级：

等级	处理
Warning	降额、上报，不立刻关断
Fault	关断相关输出，进入锁定
Critical	全部关断，必须断电或管理员清除

8.1 必须检测的故障

电源类：

- VIN 欠压。
- VIN 过压。
- 5V0 异常。
- VCC 异常。
- AVCC 异常。
- REF2V5 异常。
- -1V0 异常。

通信类：

- ADC SPI 超时。
- DAC SPI 超时。
- ADC DRDY 长时间不触发。

温度类：

- NTC 开路。
- NTC 短路。
- TEC 温度过高。
- 板载温度过高，如果有温度源。
- 温度长时间无法靠近目标，可能 TEC 接反、散热器异常。

功率类：

- LD 过流。
- LD 过压。
- LD 短路。
- LD 开路。
- TEC 过流。
- H 桥方向切换异常。
- PWM 互锁失败。

软件类：

- 看门狗复位。
- 参数 CRC 错。
- 校准参数越界。
- 状态机非法跳转。

8.2 故障锁定内容

记录：

```
typedef struct {
    uint32_t fault_code;
    uint32_t timestamp_ms;
    float vin;
    float v5;
    float avcc;
    float ref2v5;
    float temp_c;
    float tec_current;
    float ld_current;
    float ld_voltage;
    uint8_t state;
} fault_record_t;
```

故障记录要保存在 RAM 和可选 Flash 中。Flash 不要每次循环写，避免磨损。

9. 校准功能

这类板必须校准，否则 1mΩ 电流采样和精密温控很难可信。

9.1 建议校准项

参数	校准方法
ADC 零点	输入短接或零电流时采样
电流零点	输出关闭、无负载电流
电流增益	接标准电流或精密负载
DAC 零点	DAC code=0 测输出
DAC 增益	DAC 接近满量程测输出
NTC 参数	使用标准电阻或恒温点
输出电压分压	标准电压源或高精度表

9.2 参数存储

建议 Flash 参数结构：

```
typedef struct {
    uint32_t magic;
    uint16_t version;
    uint16_t length;
    float current_offset_v[2];
```

```
float current_gain_a_per_v[2];
float voltage_offset_v[2];
float voltage_gain[2];
float dac_offset_v[2];
float dac_gain[2];
ntc_cal_t ntc;
float tec_current_limit_a;
float ld_current_limit_a;
float temp_limit_c;
uint32_t crc32;
} calib_store_t;
```

启动时:

```
检查 magic
检查 version
检查 length
检查 crc32
参数范围检查
失败则加载保守默认值
```

10. 通信协议功能

建议命令至少包括:

查询:

- 读设备信息。
- 读固件版本。
- 读状态机状态。
- 读故障码。
- 读实时测量。
- 读校准参数。

控制:

- 设置 TEC 目标温度。
- 设置 TEC 电流限制。
- 启停 TEC。
- 设置 LD 电流。
- 启停 LD。
- 清故障。
- 保存参数。
- 恢复默认参数。

调试:

- 设置 DAC 原始电压。
- 读取 ADC 原始码。

- 打开假负载测试模式。
- PWM 手动测试。
- H 桥单步测试。

所有会产生功率输出的调试命令都要有权限或编译开关。

11. 任务周期建议

参考调度：

任务	周期	说明
快速电流保护	0.1-1 ms	尽可能快，必要时硬件完成
ADC 采样	1-10 ms	取决于 ADS1120 配置
LD 电流控制	1-10 ms	中速
TEC 温度 PID	50-500 ms	热系统慢
通信处理	5-20 ms	不阻塞控制
状态上报	100-1000 ms	上位机显示
参数保存	事件触发	不周期写 Flash
看门狗喂狗	主循环/RTOS	必须覆盖关键任务

12. 上电顺序建议

伪流程：

1. MCU 复位后，GPIO 立即配置为安全态
2. 关闭 PWM 输出
3. DAC 初始化，输出 0 或安全电压
4. ADC 初始化并确认 DRDY
5. 读取电源电压和温度
6. 读取电流零点
7. 如果自检通过，进入 READY
8. 收到 enable 后，开始 RAMP_UP

关断顺序：

1. 设定值 ramp 到 0
2. 关闭 Buck/H 桥 PWM
3. DAC 输出安全值
4. 等待输出电容放电
5. 进入 READY 或 OFF

故障关断顺序要更快：

1. 立即硬关 PWM/EN
2. DAC 安全值
3. 记录故障
4. 进入 FAULT_LATCHED

13. 最小可用固件版本 MVP

如果分阶段开发，建议：

阶段 1：板级 bring-up

- 时钟、GPIO、UART。
- SWD 稳定。
- LED。
- 电源电压检测。
- ADS1120 读原始码。
- DAC8562 写固定电压。

阶段 2：模拟链路验证

- DAC 输出扫描。
- ADC 通道映射。
- NTC 标准电阻换算。
- 电流采样零点读取。
- 参数打印。

阶段 3：功率级假负载

- LD Buck 假负载软启动。
- 电流限制。
- 输出电压/电流保护。
- TEC H 桥假负载方向测试。
- PWM 死区验证。

阶段 4：闭环控制

- LD 恒流。
- TEC 恒温 PID。
- 限幅、抗积分饱和。
- 方向切换 ramp。

阶段 5：产品化

- 完整故障记录。
- 参数校准和保存。
- 通信协议。
- 看门狗。
- 长时间老化测试。

- 温升和故障注入测试。

14. 调试时最有用的实时变量

建议周期上报：

```
state
fault_code
vin
v5v0
vcc
avcc
ref2v5
neg1v
ntc_temperature
tec_target_temperature
tec_current
tec_pwm_duty
tec_direction
ld_current
ld_voltage
ld_power
dac_a_voltage
dac_b_voltage
adc_raw[]
```

这些变量能让你从上位机一眼判断：

- 是测量错了，还是控制错了。
- 是电源问题，还是负载问题。
- 是硬件保护触发，还是软件状态机没放行。

15. 给嵌入式开发者的核心提醒

1. 不要把功率输出当 GPIO 玩。
2. 不要跳过 ramp。
3. 不要在故障后自动重启。
4. 不要相信第一次 ADC 读数，一定要校准和滤波。
5. 不要在 TEC 大电流时直接反向。
6. 不要让 LD 在开路状态下把输出电容充到高压后再接入。
7. 不要把模拟闭环和数字 PID 都调得很快。
8. 不要让通信命令绕过安全状态机。

这块板的软件难点不在“会不会 SPI”，而在“任何异常情况下都能先保硬件，再谈控制性能”。